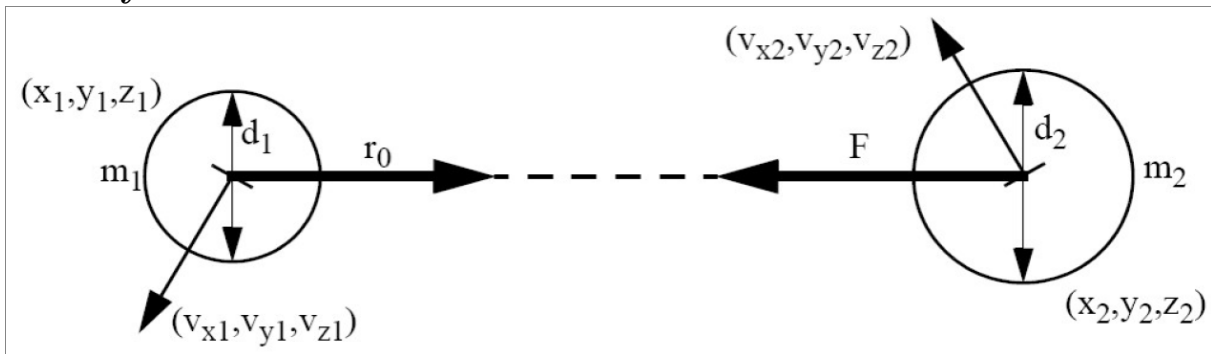


GPU Computing - Architecture and Programming

Exercise 7

- Hand in via Moodle until Wednesday, January 7, 2026, 09:00.
- Include all names on the top sheet. Hand in a *single* PDF.
- Please use naming convention: `gpu<XX>_ex<N>.{zip,tar.gz}`.
- Maximum of 4 students may collaborate.
- For programming exercises:
 - include clean and documented code
 - include a Makefile for compiling

n-Body Problem



Each object has the following data:

- Position $p = (x, y, z)$ [m]
- Velocity $v = (v_x, v_y, v_z)$ [m/s]
- Mass m [kg]

Note that calculations are performed in 3D space. These are the most important formulas:

$$\text{Force} : F = -\gamma * \frac{m_1 \cdot m_2}{r^2} \cdot \vec{r}_0 [N] \quad \gamma = 6.673 \cdot 10^{-11} \frac{Nm^2}{kg^2}$$

$$\text{Acceleration} : \vec{a} = \frac{\vec{F}}{m} [\frac{N}{kg}]$$

$$\text{Velocity} : \vec{v} = \vec{a} \cdot \Delta t [\frac{m}{s}]$$

$$\text{Distance} : \vec{s} = \vec{v} \cdot \Delta t [m]$$

With r the distance between particle 1 and 2. And $\vec{r}_0$ the unit vector between particle 1 and 2, denoting the relative direction of particle 2 to particle 1.

Ensure the following:

1. The program accepts the number of objects as command line parameter.
2. Use single precision floating points for all object data.
3. Define the length of a time step with the constant TIMESTEP (Δt).
4. Initialize positions and masses with random values, velocity with zero.
5. Perform a reasonable amount of time steps (iterations), for instance 100. Each iteration updates the velocity and position of all objects.

7.1 Naïve GPU implementation

Implement a naïve GPU version that primarily serves for testing the compute kernel. Use AOS (arrays of structures) with two packed values, as shown in the template. Do not use shared memory yet. We will later report speedups based on this baseline performance. Do not include data movement over the PCIe interface in your measurements.

- Report performance numbers for an increasing body count and discuss!

(15 points)

7.2 Optimized GPU implementation

Based on the previous implementation, perform optimizations to maximize performance for this n-body computation. In particular:

- Consider if you prefer packed SOA (structure of arrays) or AOS (arrays of structures) data structures
- Make use of tiling to optimize for shared memory use
- Optional: investigate if loop unrolling is beneficial

Report performance numbers and discuss your optimizations. For which reason(s) is a certain optimization increasing performance?

(25 points)

7.3 n-Body GPU computations and streaming

Start with the optimized GPU implementation of the n-Body problem from the last task. In this exercise we will implement a streaming version of this code, meaning that the GPU is artificially limited in terms of device memory and has to spill to host memory. In this sense, the host memory is acting as swap space. As such spilling has to be manually controlled by the user, your task is to implement a streamed version of the optimized n-Body code. Use the guidelines from the lecture, and use different streams to maximize the utilization of the GPU.

As the GPU memory capacity would result in too many bodies for reasonable run times, we artificially limit the amount of device memory to **4 MB**. Ensure in your code that you are not using more device memory! Opposed to this, run your code with a body count of **512 k**. As each body consumes about $7 \times 32\text{bit} = 28\text{B}$, better say 32B , you can see that device memory is not sufficient to completely store the problem.

Discuss the following:

- Compare your optimized streaming implementation to the non-streamed implementation (using only the default stream #0) and interpret!
- Similarly, compare against the optimized kernel from last exercise (without PCIe time) and interpret!
- Vary the segment size for the streams. Discuss the effects you are seeing!
- How many streams are required for an optimized implementation? Discuss why!

(26 points)

7.4 Willingness to present

Please declare whether you are willing to present any of the previous exercises.

The declaration can be made on a per-exercise basis. Each declaration corresponds to 50% of the exercise points. You can only declare your willingness to present exercises for which you also hand in a solution. If no willingness to present is declared you may still be required to present an exercise for which your group has handed in a solution. This may happen if as example nobody else has declared their willingness to present.

- Naïve GPU implementation (Task: 7.1)
- Optimized GPU implementation (Task: 7.2)
- n-Body GPU computations and streaming (Task: 7.3)

(33 points)

Total: 99 points